Practical – Week9

1. Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

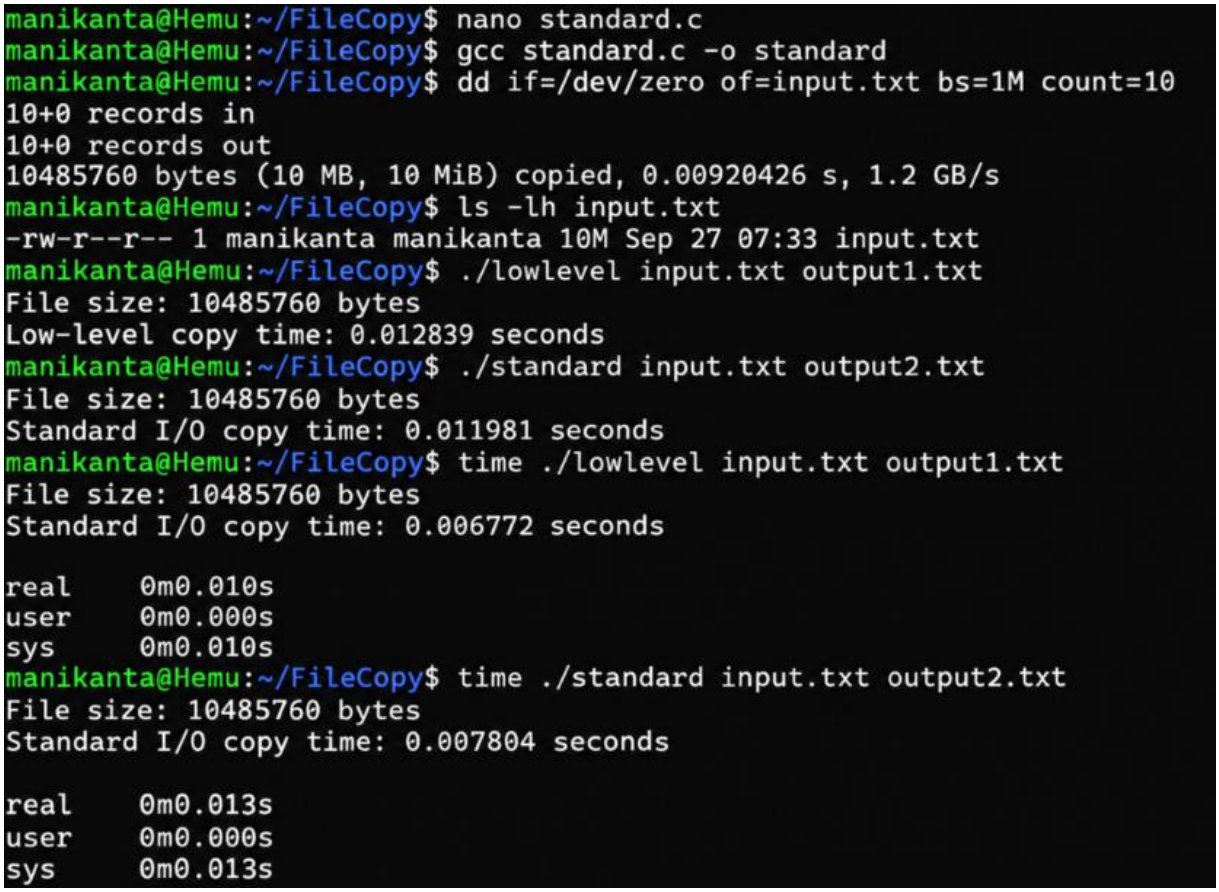
```
 GNU nano 8.7.1                          lowlevel.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>
int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }
    int src = open(argv[1], O_RDONLY);
    if (src == -1) {
        perror("Source file error");
        return 1;
    }
    int dest = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest == -1) {
        perror("Destination file error");
        close(src);
        return 1;
    }
    // Find file size using lseek()
    off_t size = lseek(src, 0, SEEK_END);
    lseek(src, 0, SEEK_SET);
    char buffer[4096];
    ssize_t bytes;
    clock_t start = clock();
    while ((bytes = read(src, buffer, sizeof(buffer))) > 0) {
        ssize_t written = 0;
        while (written < bytes) {
            ssize_t n = write(dest, buffer + written,
                              bytes - written);
            if (n <= 0) {
                perror("Write error");
                close(src);
                close(dest);
                return 1;
```

```
manikanta@Hemu:~/FileCopy$ nano lowlevel.c
manikanta@Hemu:~/FileCopy$ gcc lowlevel.c -o lowlevel
manikanta@Hemu:~/FileCopy$ ./lowlevel input.txt output1.txt
File size: 10485760 bytes
Low-level copy time: 0.005918 seconds
```

```
  GNU nano 8.7.1                        standard.c *
#include <stdio.h>
#include <time.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }
    FILE *src = fopen(argv[1], "rb");
    if (src == NULL) {
        perror("Source file error");
        return 1;
    }
    FILE *dest = fopen(argv[2], "wb");
    if (dest == NULL) {
        perror("Destination file error");
        fclose(src);
        return 1;
    }
    // Find file size using fseek()
    fseek(src, 0, SEEK_END);
    long size = ftell(src);
    fseek(src, 0, SEEK_SET);
    char buffer[4096];
    size_t bytes;
    clock_t start = clock();
    while ((bytes = fread(buffer, 1, sizeof(buffer), src)) > 0) {
        if (fwrite(buffer, 1, bytes, dest) != bytes) {
            perror("Write error");
            fclose(src);
            fclose(dest);
            return 1;
        }
    }

    if (ferror(src)) {
        perror("Read error");
```



Report: The file was successfully copied using open(), read(), and write() system calls and fopen(), fread(), and fwrite() library functions. The cmp command verified that both output files matched the original, and the time command was used to compare their execution times.

2. Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```
  GNU nano 8.7.1                          redirect.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int fd_in, fd_out;
    // Open input file
    fd_in = open("input.txt", O_RDONLY);
    if (fd_in < 0) {
        perror("Input file error");
        return 1;
    }
    // Open output file
    fd_out = open("output.txt",
                  O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd_out < 0) {
        perror("Output file error");
        close(fd_in);
        return 1;
    }
    // Redirect standard input
    dup2(fd_in, STDIN_FILENO);
    // Redirect standard output
    dup2(fd_out, STDOUT_FILENO);
    close(fd_in);
    close(fd_out);
    // Execute a command using redirected I/O
    execlp("wc", "wc", "-l", (char *)NULL);
    perror("execlp failed");
    return 1;
}
```

```
manihanta@Hemu:~/OS_Lab$ nano redirect.c
manikanta@Hemu:~/OS_Lab$ gcc redirect.c -o redirect
manikanta@Hemu:~/OS_Lab$ echo -e "Hello\nWelcome to Linux\nOperating Systems" > input.txt

manikanta@Hemu:~/OS_Lab$ cat input.txt
Hello
Welcome to Linux
Operating Systems

manikanta@Hemu:~/OS_Lab$ ./redirect
manikanta@Hemu:~/OS_Lab$ cat output.txt
3
```